```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("sales_data.csv")
 df.info()
```

Output: <class 'pandas.DataFrame'>

```
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 25 columns):
#   Column            Non-Null Count  Dtype
---  ------           --------------  -----
0   ORDERNUMBER       2823 non-null   int64
1   QUANTITYORDERED   2823 non-null   int64
2   PRICEEACH         2823 non-null   float64
3   ORDERLINENUMBER   2823 non-null   int64
4   SALES             2823 non-null   float64
5   ORDERDATE         2823 non-null   str
6   STATUS            2823 non-null   str
7   QTR_ID            2823 non-null   int64
8   MONTH_ID          2823 non-null   int64
9   YEAR_ID           2823 non-null   int64
10  PRODUCTLINE       2823 non-null   str
11  MSRP              2823 non-null   int64
12  PRODUCTCODE       2823 non-null   str
13  CUSTOMERNAME      2823 non-null   str
14  PHONE             2823 non-null   str
15  ADDRESSLINE1      2823 non-null   str
16  ADDRESSLINE2      302 non-null    str
17  CITY              2823 non-null   str
18  STATE             1337 non-null   str
19  POSTALCODE        2747 non-null   str
20  COUNTRY           2823 non-null   str
21  TERRITORY         1749 non-null   str
22  CONTACTLASTNAME   2823 non-null   str
23  CONTACTFIRSTNAME  2823 non-null   str
24  DEALSIZE          2823 non-null   str
dtypes: float64(2), int64(7), str(16)
memory usage: 551.5 KB
```

👉 Important business rule

- For Sales Analysis revenue, trends, products are important
- These columns STATE POSTALCODE TERRITORY are NOT critical
- Missing values will NOT break sales calculations
- So DO NOT delete rows blindly.

STEP 1: Check Missing Values Properly

```
df.isnull().sum()
```

Output:

```
ORDERNUMBER        0
QUANTITYORDERED    0
PRICEEACH          0
ORDERLINENUMBER    0
SALES              0
ORDERDATE          0
STATUS             0
```

```
QTR_ID              0
MONTH_ID            0
YEAR_ID             0
PRODUCTLINE         0
MSRP                0
PRODUCTCODE         0
CUSTOMERNAME        0
PHONE               0
ADDRESSLINE1        0
ADDRESSLINE2        2521
CITY                0
STATE               1486
POSTALCODE          76
COUNTRY             0
TERRITORY           1074
CONTACTLASTNAME     0
CONTACTFIRSTNAME    0
DEALSIZE            0
dtype: int64
```

Note:- This confirms exactly how many values are missing.

STEP 2: Decide What To Do (Business Logic)

OPTION A (Recommended for beginners): **Fill Missing Values**

This is safest and interview-friendly.

```
df['STATE'] = df['STATE'].fillna('Unknown')
df['POSTALCODE'] = df['POSTALCODE'].fillna('Unknown')
df['TERRITORY'] = df['TERRITORY'].fillna('Unknown')
df['ADDRESSLINE2'] = df['ADDRESSLINE2'].fillna('Unknown')
```

- Keeps all 2823 records
- No effect on revenue analysis
- Easy to explain in interviews

OPTION B (Only if doing regional analysis): Drop rows

❌ Not recommended now, but FYI:

```
df.dropna(subset=['STATE', 'POSTALCODE', 'TERRITORY'], inplace=True)
```

Notes

What does FYI mean? FYI = "For Your Information"

👉 Meaning:

- "I'm telling you this so you are aware"
- "You don't need to use this now"
- *"Just knowledge for future situations"*

Example in simple words: "This is not required right now, but you should know it exists."

So when I said **FYI**, I meant: ❌ Don't apply this now
✅ Just understand what it does

2️⃣ What does this code actually do?

df.dropna(subset=['STATE', 'POSTALCODE', 'TERRITORY'], inplace=True)

Let's break it down line by line 👇

🔹 dropna()

- Removes rows that contain **missing (NaN) values**

🔹 subset=[...]

- Apply the rule **only to these columns**
- Here:
  - STATE
  - POSTALCODE
  - TERRITORY

🔹 inplace=True

- Permanently modifies the dataframe
- No copy is kept

3️⃣ What happens AFTER running this?

👉 Any row that has even ONE missing value in:

- STATE **OR**
- POSTALCODE **OR**
- TERRITORY

❌ That entire row is **deleted**

Example (Very Simple)

STATE	POSTALCODE	TERRITORY	SALES
CA	90001	NA	100
NaN	75001	EMEA	200
NY	**NaN**	NA	300

Note:- Here NA is north America

After dropna(...) ❌👇

STATE	POSTALCODE	TERRITORY	SALES
CA	90001	NA	100

👉 2 rows are gone forever

4️⃣ Why is this ❌ NOT recommended for you now?

🔴 Reason 1: You lose sales data

Sales = MONEY
Deleting rows = deleting revenue

If a row has: SALES = 50,000
STATE = NaN

Note:- 👉 Dropping it **kills valid business data**

🔴 **Reason 2: Your project goal is NOT regional analysis**

Your project goal is:

- Monthly sales
- Revenue trends
- Top products

❌ None of these depend on:

- STATE
- POSTALCODE
- TERRITORY

So dropping rows is **unnecessary damage**

5️⃣ **When DOES Option B make sense? (Important)**

✅ **Use dropna() ONLY when:**

✓ Your analysis **depends on geography**
✓ You need **accurate regional mapping**
✓ Missing location data **breaks analysis**

Example cases:

- Sales by **state**
- Sales heatmap by **territory**
- Logistic optimization by **postal code**

Then you can say:

"Since regional accuracy was critical, I removed rows with missing geographic attributes."

6️⃣ **Why Option A (fillna) is safer?**

```
df['STATE'].fillna('Unknown', inplace=True)
```

Benefits:

✓ Keeps all sales records
✓ Revenue numbers stay correct
✓ You can still group "Unknown" separately
✓ Interview-safe approach

7️⃣ **Interview Question They MAY Ask You**

Q: Why didn't you drop missing STATE values?

Perfect Answer:

"Because those missing values were in non-critical columns. Dropping them would result in loss of valid sales data, so I used imputation instead."

🔥 This is **analyst-level thinking**

8 Final One-Line Summary

- **FYI** = just for knowledge
- dropna() = deletes rows permanently
- Use it **only when missing data breaks analysis**
- For sales trends → **never delete revenue rows casually**

STEP 3: Verify Fix

```
df.isnull().sum()
```

You should see 0 missing values now.

Output:

```
ORDERNUMBER        0
QUANTITYORDERED    0
PRICEEACH          0
ORDERLINENUMBER    0
SALES              0
ORDERDATE          0
STATUS             0
QTR_ID             0
MONTH_ID           0
YEAR_ID            0
PRODUCTLINE        0
MSRP               0
PRODUCTCODE        0
CUSTOMERNAME       0
PHONE              0
ADDRESSLINE1       0
ADDRESSLINE2       0
CITY               0
STATE              0
POSTALCODE         0
COUNTRY            0
TERRITORY          0
CONTACTLASTNAME    0
CONTACTFIRSTNAME   0
DEALSIZE           0
dtype: int64
```

STEP 4: Correct Way (Use ORDERDATE)

Convert ORDERDATE from string → datetime

```
df['ORDERDATE'] = pd.to_datetime(df['ORDERDATE'])
df.info()
```

Output:

```
<class 'pandas.DataFrame'>
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 25 columns):
#   Column        Non-Null   Count    Dtype
---  ------        --------   ------   -----
.....  ......      ........   ........ ......
5   ORDERDATE     2823       non-null datetime64[us]
...  ........      .....     ........ ......
dtypes: float64(2), int64(7), str(16)
memory usage: 551.5 KB
```

If You Do NOT Convert

If it stays as string:

- Sorting will be wrong (string sorting, not date sorting)
- Cannot use .dt functions
- Cannot do time-series analysis
- ML model won't understand it properly

STEP 5: Create Month & Year (If Needed)

Even though you already have MONTH_ID and YEAR_ID, this shows **real Pandas skill**:

```
df['MONTH'] = df['ORDERDATE'].dt.month
df['YEAR'] = df['ORDERDATE'].dt.year
```

(You can also directly use MONTH_ID and YEAR_ID — both are valid)

Q). What is .dt in pandas?

- .dt is a datetime accessor in pandas.
- It allows you to extract date-related information from a datetime column.

Check:- df.info()

Output: <class 'pandas.DataFrame'>

```
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 25 columns):
#   Column        Non-Null Count  Dtype
---  ------        --------------  -----
...  ........      .....           ......
25  MONTH         2823 non-null   int32
26  YEAR          2823 non-null   int32
dtypes: float64(2), int64(7), str(16)
memory usage: 551.5 KB
```

STEP 6: Monthly Sales Analysis (Correct Way for Your Data)

Using existing MONTH_ID & YEAR_ID (Best)

```
monthly_sales = df.groupby(['YEAR_ID', 'MONTH_ID'])['SALES'].sum()
monthly_sales
```

YEAR_ID	MONTH_ID	
2003	1	129753.60
	2	140836.19
	3	174504.90
	4	201609.55
	5	192673.11
	6	168082.56
	7	187731.88
	8	197809.30
	9	263973.36
	10	568290.97
	11	1029837.66
	12	261876.46
2004	1	316577.42
	2	311419.53
	3	205733.73
	4	206148.12
	5	273438.39
	6	286674.22
	7	327144.09
	8	461501.27

```
            9        320750.91
           10        552924.25
           11       1089048.01
           12        372802.66
2005        1        339543.42
            2        358186.18
            3        374262.76
            4        261633.29
            5        457861.06
Name: SALES, dtype: float64
```

Notes

1️⃣ 👉 Group the data by **Year + Month combination**

Example:

YEAR_ID	MONTH_ID	SALES
2022	1	500
2022	1	700
2022	2	300
2023	1	900

After grouping,

```
monthly_sales = df.groupby(['YEAR_ID', 'MONTH_ID'])['SALES'].sum()
monthly_sales
```

2️⃣ **['SALES'] ,** We are selecting only the **SALES column** for aggregation.

3️⃣ **.sum(),** For each (Year, Month) group → add total sales.

Output:

```
YEAR_ID  MONTH_ID
2022            1     1200
                2      300
2023            1      900
Name: SALES, dtype: float64
```

STEP 7: Top Product Analysis

```
top_products = df.groupby('PRODUCTLINE')['SALES'].sum().sort_values(ascending=False)
top_products
```

```
PRODUCTLINE
Classic Cars        3919615.66
Vintage Cars        1903150.84
Motorcycles         1166388.34
Trucks and Buses    1127789.84
Planes               975003.57
Ships                714437.13
Trains               226243.47
Name: SALES, dtype: float64
```

📊 **Business Insight:** Which product brings **maximum money**

Notes

1 df.groupby('PRODUCTLINE')

👉 Groups the data by product category.

Example:

PRODUCTLINE	SALES
Classic Cars	500
Motorcycles	300
Classic Cars	700
Planes	200

After grouping:

- Classic Cars
- Motorcycles
- Planes

2 ['SALES'].sum()

For each product category → calculate total sales.

Result:

PRODUCTLINE	SALES
Classic Cars	1200
Planes	200
Motorcycles	300

3 .sort_values(ascending=False)

Sort from **highest to lowest sales**.

PRODUCTLINE	SALES
Classic Cars	1200
Motorcycles	300
Planes	200

Highest selling product line appears at top.

🎯 Final Meaning

👉 This code finds **Top Performing Product Categories based on Total Sales**.

STEP 8: Revenue Trend

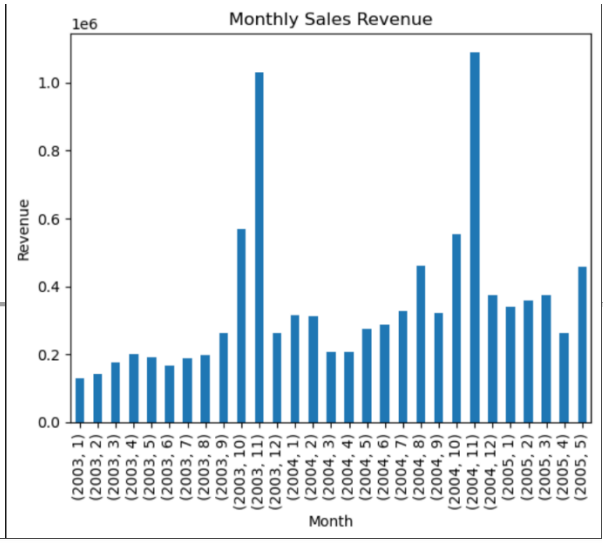
```
daily_sales = df.groupby('ORDERDATE')['SALES'].sum()
daily_sales
```

```
ORDERDATE
2003-01-06    12133.25
2003-01-09    11432.34
2003-01-10     6864.05
2003-01-29    54702.00
2003-01-31    44621.96
                ...
2005-05-13    31821.90
2005-05-17    59475.10
2005-05-29    51233.18
2005-05-30    14578.75
```

```
2005-05-31    78918.03
Name: SALES, Length: 252, dtype: float64
```

Shows:

- Growth
- Decline
- Seasonal patterns



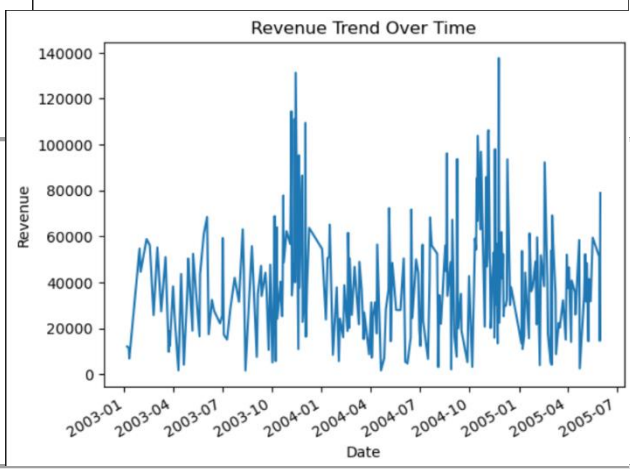
STEP 9: Visualization (Matplotlib)

📊 Bar Chart – Monthly Revenue

```
monthly_sales.plot(kind='bar')
plt.title("Monthly Sales Revenue")
plt.xlabel("Month")
plt.ylabel("Revenue")
plt.show()
```

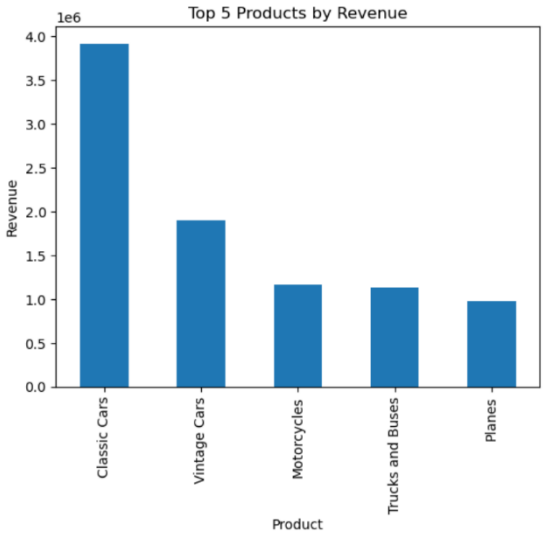
📈 Line Chart – Revenue Trend

```
daily_sales.plot()
plt.title("Revenue Trend Over Time")
plt.xlabel("Date")
plt.ylabel("Revenue")
plt.show()
```



📦 Bar Chart – Top Products

```
top_products.head(5).plot(kind='bar')
plt.title("Top 5 Products by Revenue")
plt.xlabel("Product")
plt.ylabel("Revenue")
plt.show()
```



Interview-Ready Explanation (Very Important)

You can confidently say:

"The dataset had missing values in STATE, POSTALCODE, and TERRITORY.
Since these columns were not critical for sales calculations, I handled them by imputing 'Unknown'.
I converted ORDERDATE from string to datetime and used it for time-based sales analysis."

OR

"I analyzed sales data from a CSV using Pandas and NumPy.
I calculated monthly revenue, identified top-performing products using groupby aggregations, and visualized revenue trends using Matplotlib."

🔥 This sounds **professional + analytical**

Interview Questions Based on This Project

Q1. What was the objective of your sales data analysis project?

Answer: The objective was to analyze sales data from a CSV file to identify monthly revenue trends, top-performing product lines, and overall sales performance using Pandas, NumPy, and Matplotlib.

Q2. How did you load and inspect the dataset?

Answer: I loaded the dataset using pd.read_csv() and inspected its structure using df.head() and df.info() to understand data types, missing values, and column relevance.

Q3. Did your dataset contain missing values? How did you handle them?

Answer: Yes, columns like STATE, POSTALCODE, and TERRITORY had missing values. Since these columns were not critical for revenue calculations, I filled missing values with a constant value like 'Unknown' to avoid losing valid sales records.

Q4. Why didn't you drop rows with missing values?

Answer: Dropping rows would have resulted in loss of valid sales and revenue data. Since the missing values were in non-critical geographic columns, imputation was a safer business decision.

Q5. Explain the difference between NA and NaN in your dataset.

Answer: NA represented North America, which is a valid category value, whereas NaN represents missing data. Pandas only treats NaN as missing.

Q6. How did you perform monthly sales analysis?

Answer: I grouped the data by YEAR_ID and MONTH_ID and calculated total sales using the groupby() and sum() functions.

Q7. How did you identify top-performing products?

Answer: I grouped the dataset by PRODUCTLINE, aggregated total sales, and sorted the results in descending order to identify top product lines.

Q8. How did you analyze sales trends over time?

Answer: I converted ORDERDATE into datetime format and grouped sales by date to visualize revenue trends using a line chart.

Q9. What visualizations did you create and why?

Answer: I created bar charts for monthly sales and top product lines, and a line chart for revenue trends to clearly communicate business insights.

Q10. What Pandas best practices did you follow?

Answer: I avoided chained assignments, used explicit column reassignment, handled missing values carefully, and ensured correct data types for time-series analysis.

Q11. How can this project be extended?

Answer: It can be extended by adding regional analysis, Power BI dashboards, SQL-based analysis, or predictive forecasting models.

Q: As we use in code pd for pandas, but why we havn't use np?

Very smart question 👏🔥, This shows you're thinking properly.
You're asking: *We use pd for pandas, but why didn't we use np for NumPy?*

📌 **Short Answer**

Because **your current operations don't require NumPy**.

You are doing:

- read_csv
- groupby
- sum
- fillna
- to_datetime
- sort_values

All of these are **pandas functions**, not NumPy functions.

🧠 **What is pd and np?**

import pandas as pd
import numpy as np

- pd → alias for **pandas**
- np → alias for **NumPy**

They are two different libraries.

🔍 **When Do We Use np?**

We use NumPy for:

✅ **Mathematical operations**

- np.mean()
- np.median()
- np.sqrt()
- np.log()

✅ **Creating arrays:** np.array([1,2,3])

✅ **Random numbers:** np.random.rand(5)

✅ **Advanced numerical computations**

📊 Why You Didn’t Need np Here

Because pandas already internally uses NumPy.
For example:
df['SALES'].sum()

Internally, pandas is using NumPy functions.

So you don’t need to call np manually.

🧠 Important Concept

👉 Pandas is built on top of NumPy.

Think like this:

NumPy → handles numerical engine
Pandas → handles tables (DataFrames)